Guion de la defensa (15 minutos) · Alpha Centauri

Antes de empezar (10 minutos antes): abrir https://alpha-centauri-five.vercel.app/#/monitoreo y pulsar **Medir latencia ahora** para despertar la API y cargar los tres modelos; abrir https://alpha-centauri-mlflow.onrender.com para despertar MLflow. Tener a mano el repositorio y `docs/informe-final.md`.

Minuto	Qué se muestra	Qué se dice	Quién
0–1	`/landing/`	Misión de la ficha, los tres niveles, qué entregamos: ambiente, modelos, pipeline, API, web, rollback.	MJ
1–4	Portada `#/` (cohete)	Replay del recomendado de N3 bajo la pasarela con viento. Cambiar con las pestañas a N1 y a N2: cómo crece la complejidad. Modo *replay* visible; criterios de aterrizaje en la consola.	Isaías
4–6	`#/experimentos`	Tabla de complejidad: qué añade cada nivel, observación, obstáculos, viento, y cuánto pierde el modelo del nivel anterior sin reentrenar (99 → 5 %, 100 → 93,5 %). Pestaña del experimento 5: variables en negrita, decisiones del pipeline, tres semillas.	MJ
6–8	Ficha de `ppo-pasarela-transfer-1m5-v0.4-s2`	Identidad: commit, SHA-256, manifiesto de la sección 11, versión en MLflow (abrir la run). Hipótesis y decisión. **Evaluación en vivo**: suite v1, 6 épocas; luego el mismo modelo en N3 v1 (misma familia): se ve la degradación.	Rojas
7–8	`#/vivo`	**Viento en vivo**: lanzar una misión con el recomendado de N3 bajo la pasarela, subir el viento a 20 m/s a mitad de vuelo y ver cómo corrige; es la perturbación del nivel 3 aplicada a mano, con el modelo congelado y la API simulando paso a paso.	Isaías
8–10	`#/incidente`	Incidente medido para las tres semillas, intervención (cuatro ejecuciones, criterio fijado antes), recomendado de N3 con 85,5 % con viento.	Rojas
10–12	`#/monitoreo` + terminal	Qué commit y catálogo se sirven, alertas, últimas decisiones del pipeline y evidencia JSON. **Rollback en vivo**: `python catalogo.py restaurar --nivel N3 --motivo "demo"`, commit y push; mientras despliega, enseñar la evidencia del rollback anterior; al volver la API, `GET /version` muestra el catálogo nuevo (si el tiempo no alcanza, se enseña la evidencia y se deja el comando preparado).	MJ
12–13	Pipeline	`pipelines/README.md`: ocho etapas y criterio; una evidencia abierta; captura de Prefect.	Rojas

Minuto	Qué se muestra	Qué se dice	Quién
13–14	Límites	Semillas y medias, ejecuciones con `*`, plan gratuito, `mlflow.db` en git, qué haríamos después (abertura aleatoria, torque).	Isaías
14–15	Cierre	Contribuciones y preguntas.	todos

Preguntas probables y respuesta corta:

- **¿Por qué el recomendado no es la media?** El pipeline promueve con un criterio fijo (+2 puntos, viento, colisiones) y conserva el anterior; la media de las tres semillas está en el informe y en las fichas. El test reservado da −1,5, −1 y −0,5 puntos respecto a la suite v1: no hay sobreajuste a la suite.
- **¿Qué pasa si el modelo nuevo falla en producción?** `catalogo.py restaurar`, commit y push: la API sirve el anterior en unos tres minutos; `comprobar_api.py` deja evidencia. Hecho dos veces con el nivel 3.
- **¿Por qué cambiaron las torres por la pasarela?** Decisión del equipo tras medir: las torres solo obligaban a desviarse en el 30 % de las salidas; la pasarela, siempre. Los modelos de torres siguen reproducibles.
- **¿`terminated` o `truncated`?** Éxito y fallos de misión terminan (`terminated`); agotar los 50 pasos trunca (`truncated`) con la misma penalización. Pruebas en `tests/test_ambiente.py`.
- **¿Qué es Python o TypeScript?** Python es la autoridad: ambiente, modelos y evaluación. La web solo muestra lo que devuelve la API y reproduce trayectorias grabadas.
- **¿Por qué Prefect en otro entorno?** Prefect 3.8 no convive con la versión de FastAPI de la API; el orquestador tiene su `requirements-pipeline.txt` y ejecuta las etapas con el intérprete del proyecto.